

Entity Authentication

Entity Authentication

- An **entity** can be a person, a process, a client, or a server.
- The entity whose identity needs to be proved is called the **claimant**
- The party that tries to prove the identity of the claimant is called the **verifier**.
- **Entity authentication** is a technique designed to let one party prove the identity of another party.
 - Popularly used in **Key Management**

Data-Origin Vs Entity Authentication

- Message authentication (or data-origin authentication) might not happen in real time; entity authentication does.
 - Data-origin authentication is required when an email is sent from Alice to Bob.
 - Entity authentication is required when Alice gets cash from an automatic teller machine.
- Message authentication needs to be repeated for each new message
 - Entity authentication authenticates the claimant for the entire duration of a session.

Verification Methods in Entity Authentication

- The claimant must identify herself to the verifier in one of three kinds of witnesses:
 - **Something known**
 - Password, a PIN, a secret key,
 - **Something possessed**
 - Passport, a driver's license, an identification card, a credit card, and a smart card
 - **Something inherent**
 - Some inherent characteristic of the claimant.
 - Examples are conventional signatures, fingerprints, voice, facial characteristics, retinal pattern, and handwriting

Classification of Entity Authentication Schemes

- Password based authentication
- Challenge-Response based authentication
- Zero-knowledge authentication

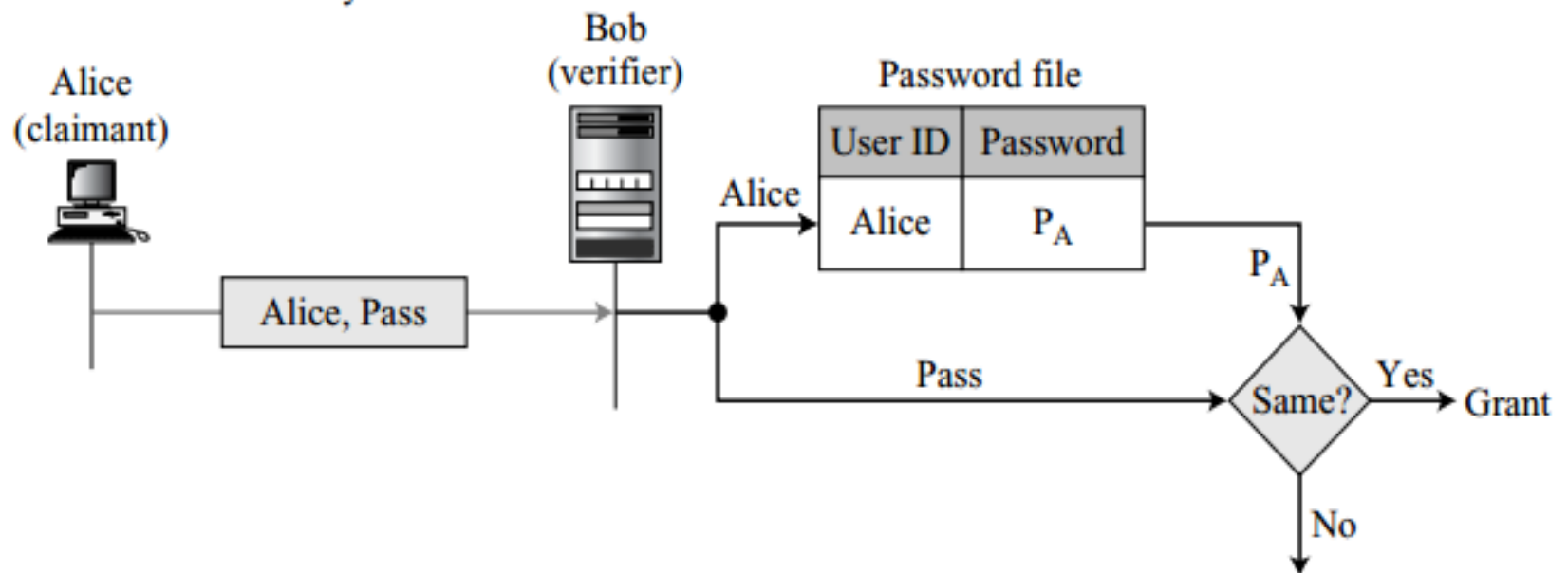
Password-based Authentication

- A **password** is used when a user needs to access a system to use the system's resources (**login**).
- Password-based Authentication is divided into two groups:
 - fixed password
 - one-time password

Fixed Password

- It is a password that is used over and over again for every access.
- Four approaches have been built on fixed passwords
- **First Approach (conventional):**
 - the system keeps a table (a file) that is sorted by user identification.
 - the user sends her user identification and password, in plaintext, to the system.
 - The system uses the identification to find the password in the table.

P_A : Alice's stored password
Pass: Password sent by claimant

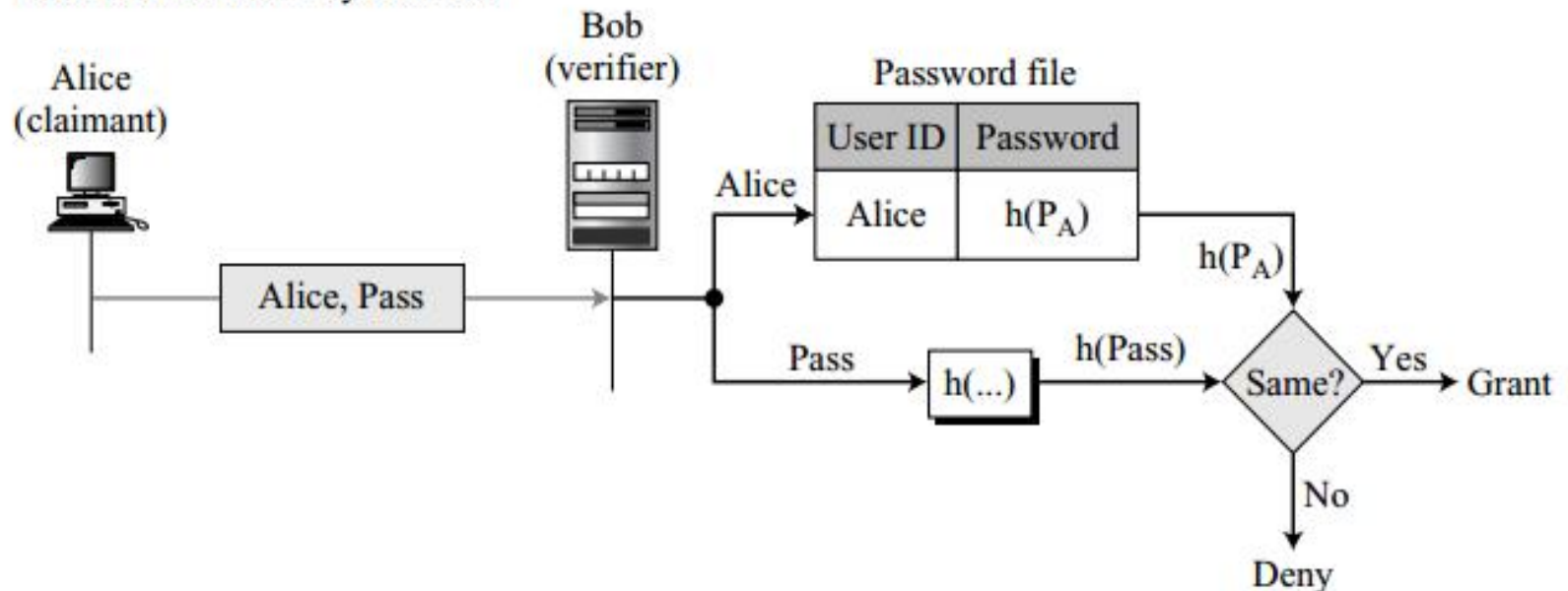


- Attacks on the First Approach:
 - Eavesdropping
 - Stealing a password
 - Accessing a password file
 - Guessing attack

- Second Approach of fixed password: Hashing the password
- A more secure approach is to store the hash of the password (instead of the plaintext password) in the password file. (susceptible to dictionary attack)

P_A : Alice's stored password

Pass: Password sent by claimant



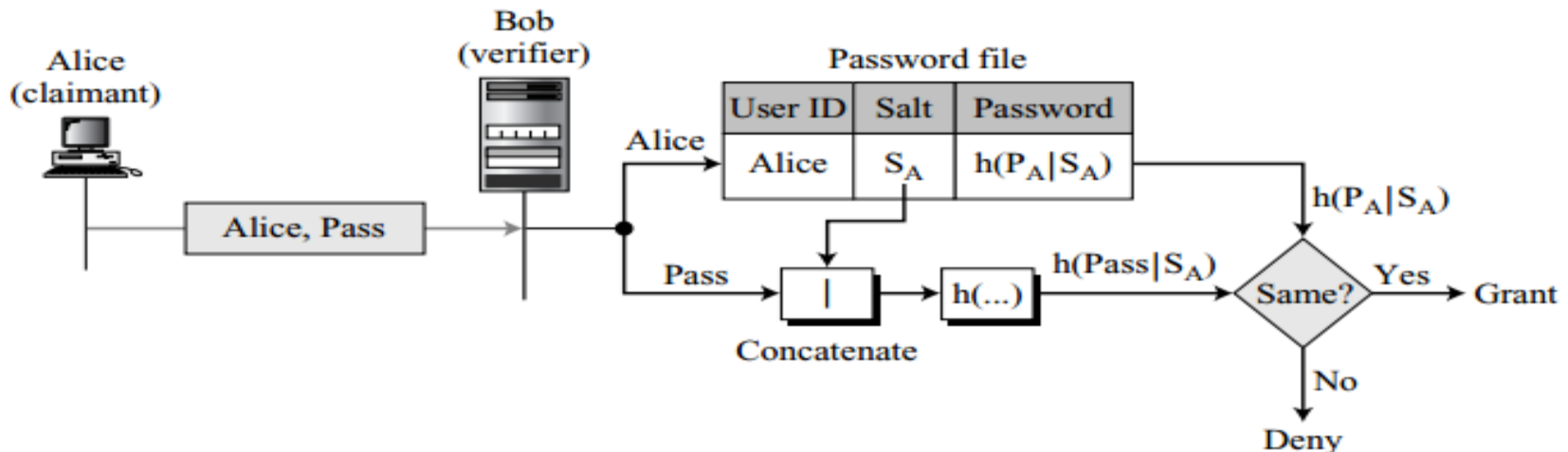
- **Third Approach of fixed password: Salting the password**

- Salt is a random string is concatenated to the password
- The salted password is then hashed.

P_A : Alice's password

S_A : Alice's salt

Pass: Password sent by claimant



- Fourth Approach of fixed password: Hybrid
 - two identification techniques are combined
 - Example: use of an ATM card with a PIN

One-Time Password

- A one-time password is a password that is used only once.
- Three approaches are used:
- **First Approach:**
 - the user and the system agree upon a list of passwords (Each password on the list can be used only once)
 - the system and the user must keep a long list of passwords
 - if the user does not use the passwords in sequence, the system needs to perform a long search to find the match.

One-Time Password

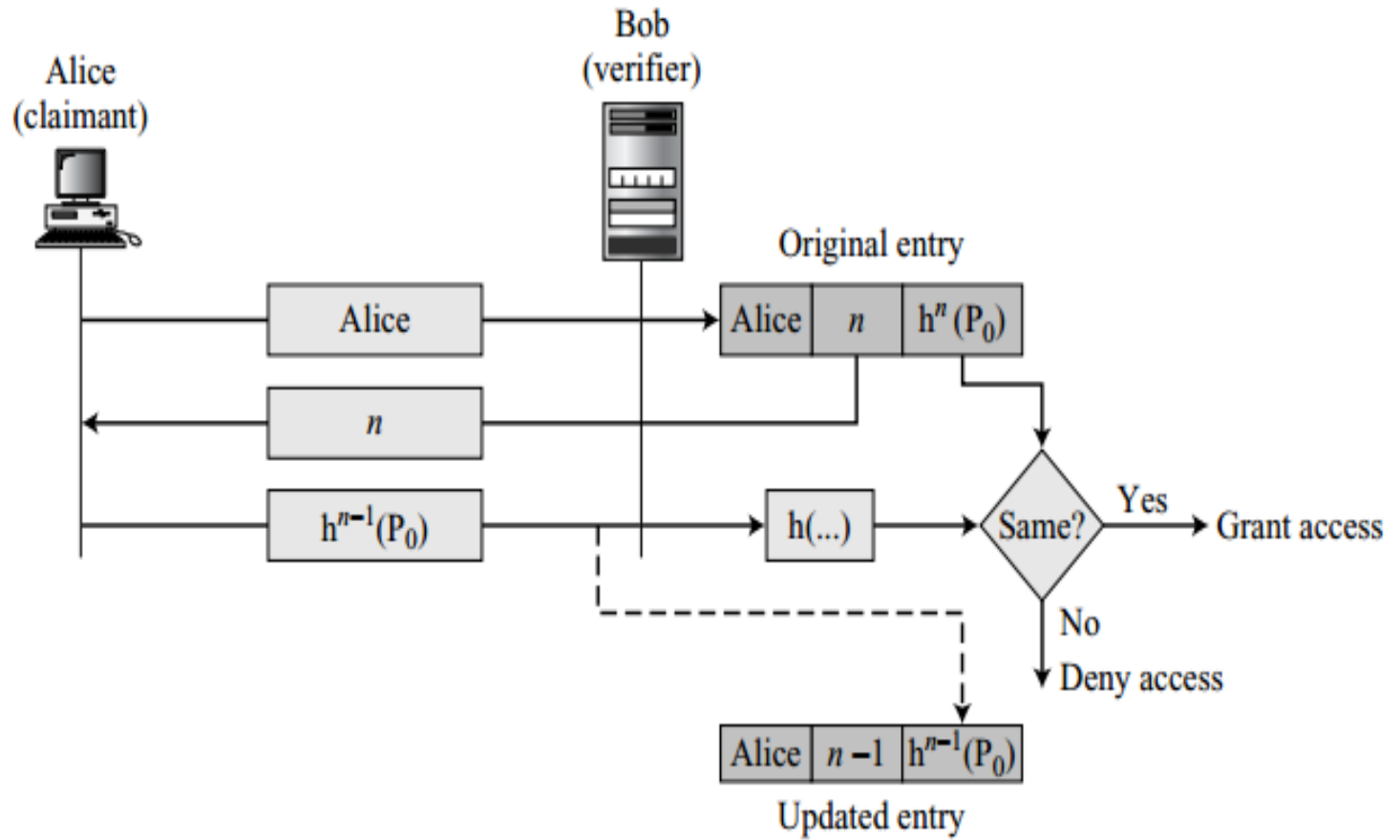
- **Second Approach:**
 - the user and the system agree to sequentially update the password.
 - The user and the system agree on an original password, P_1 , which is valid only for the first access
 - During the first access, the user generates a new password, P_2 , and encrypts this password with P_1 as the key.
 - P_2 is the password for the second access.
 - P_i is used to create P_{i+1}

One-Time Password

- Third Approach: Lamport one-time password
 - the user and the system create a sequentially updated password using a hash function
 - the user and the system agree upon an original password, P_o , and a counter, n .
 - The system calculates $h^n(P_o)$, where h^n means applying a hash function n times.

$$h^n(x) = h(h^{n-1}(x)) \quad h^{n-1}(x) = h(h^{n-2}(x)) \quad \dots \quad h^2(x) = h(h(x)) \quad h^1(x) = h(x)$$

Lamport one-time password



Challenge-Response Authentication

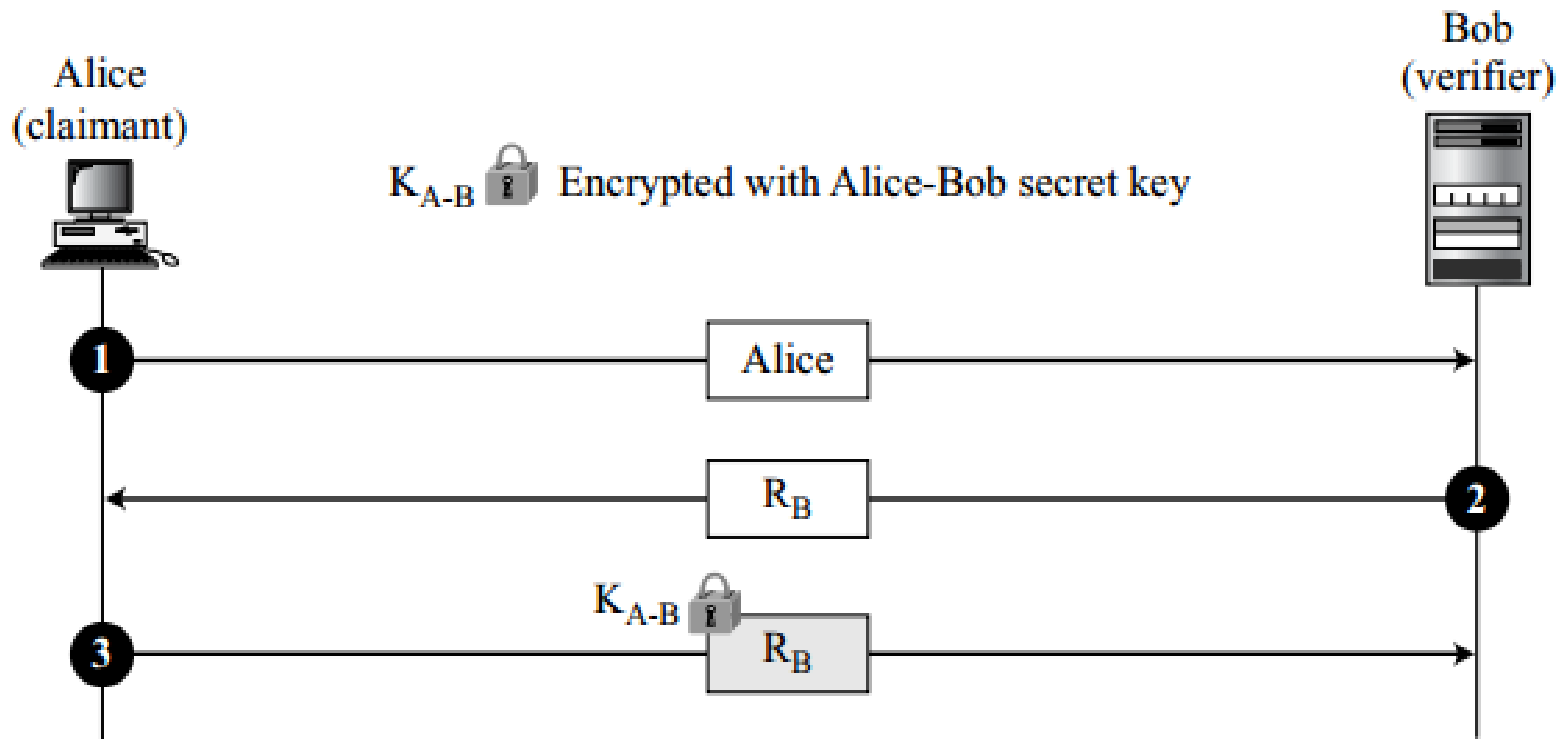
- In challenge-response authentication, the claimant proves that she knows a secret without sending it to the verifier.
- The **challenge** is a time-varying value such as a random number or a timestamp that is sent by the verifier.
- The claimant applies a function to the challenge and sends the result, called a **response**, to the verifier.

Challenge-response authentication Methods

- Using a Symmetric-Key Cipher
- Using Keyed-Hash Functions
- Using an Asymmetric-Key Cipher
- Using Digital Signature

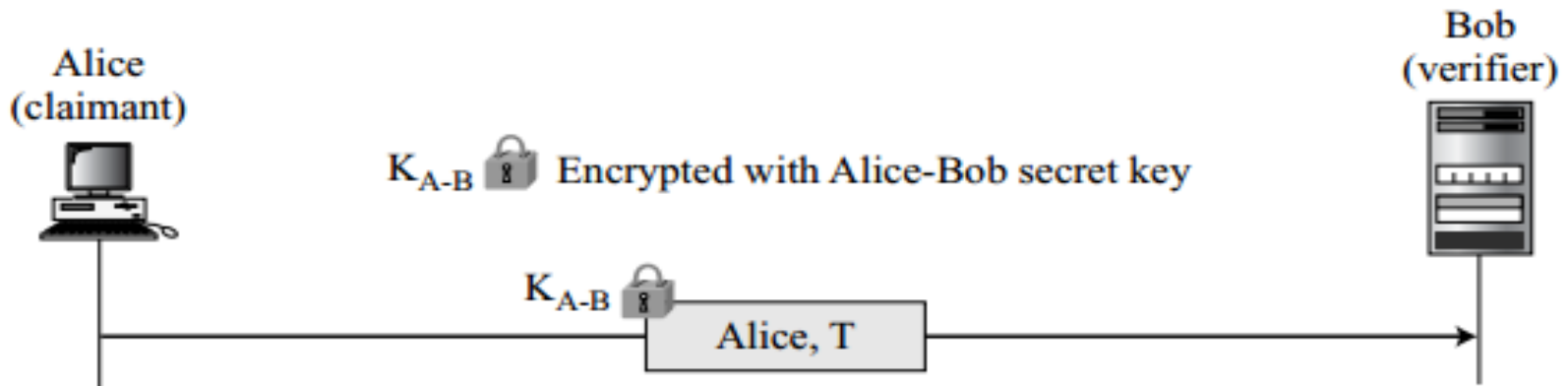
Challenge-response authentication using Symmetric-Key Cipher

- **First Approach: Using nonce**
 - **Nonce** is a time-varying random number used only once
 - unidirectional



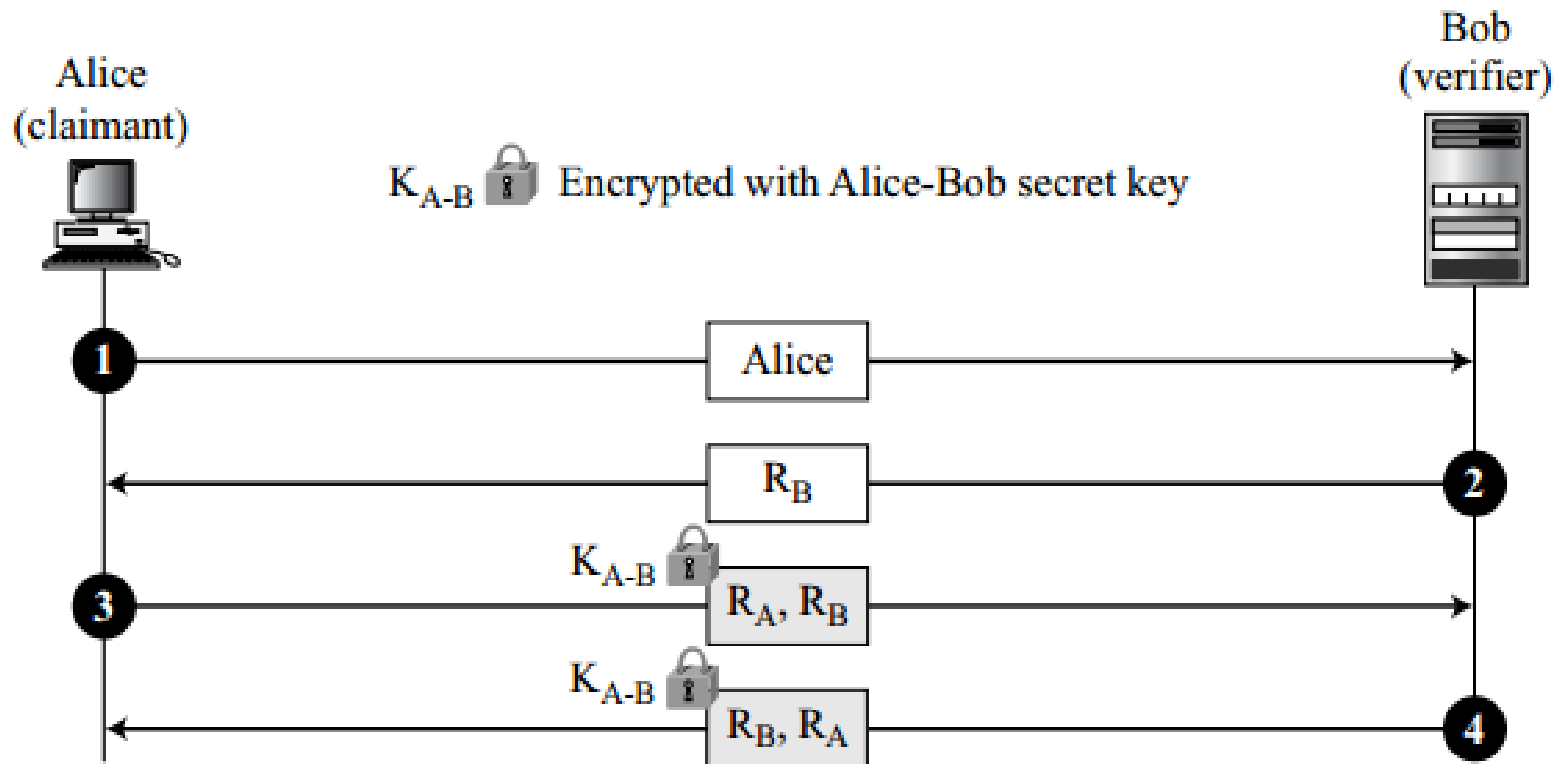
Challenge-response authentication using Symmetric-Key Cipher

- **Second Approach: Using timestamp**
 - A **timestamp** is a time-varying value which changes with time
 - Unidirectional
 - client and the server clocks are synchronized



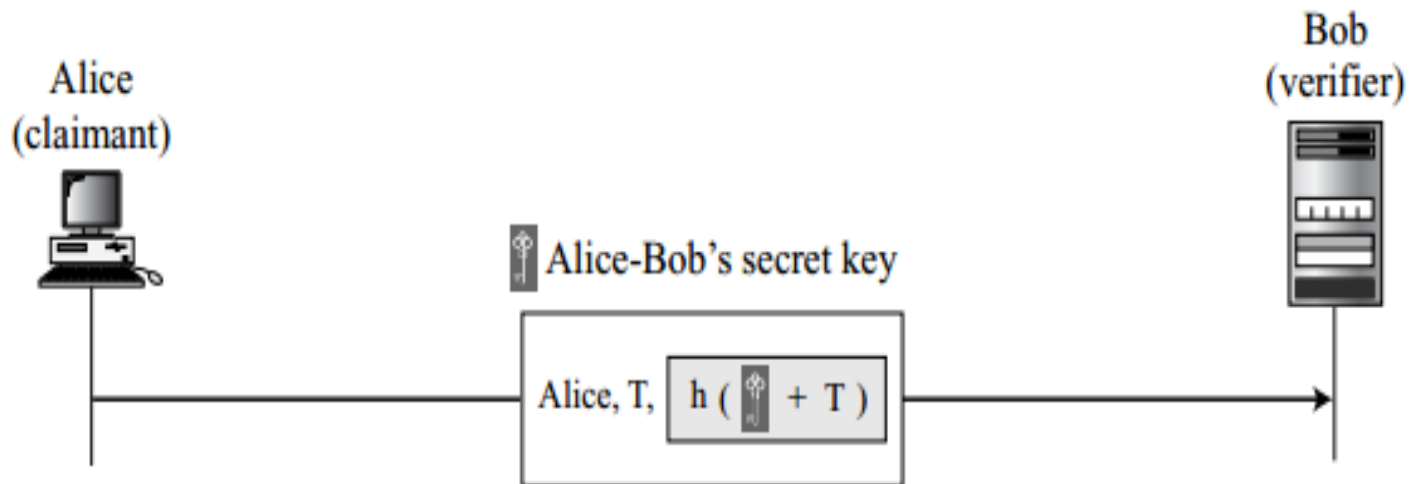
Challenge-response authentication using Symmetric-Key Cipher

- **Third Approach: Using** bidirectional authentication



Challenge-response authentication using Keyed-Hash Functions

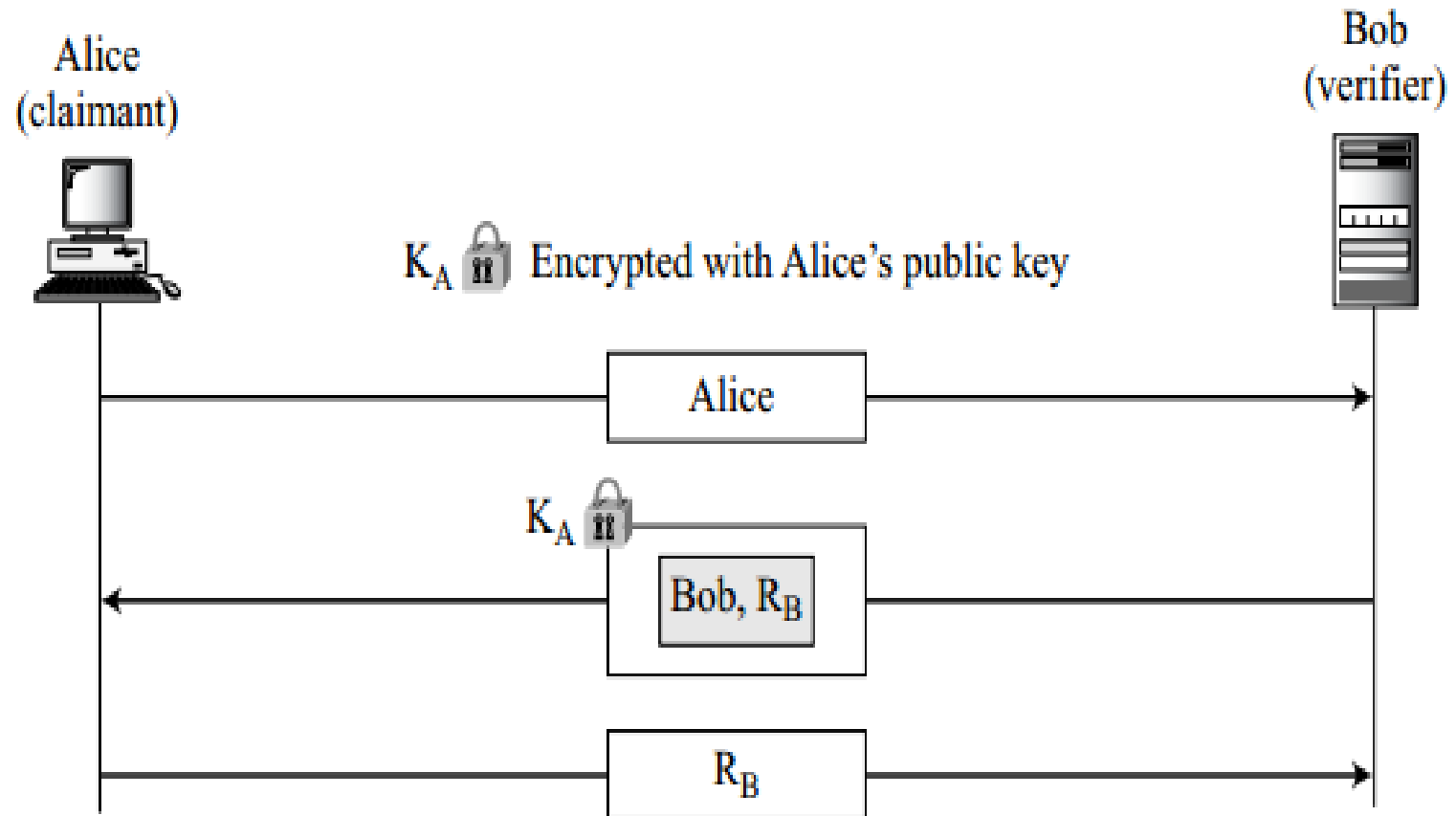
- Using a **keyed-hash function (MAC)** preserves the integrity of challenge and response messages and at the same time uses a secret, the key.



Challenge-response authentication using Asymmetric-Key Cipher

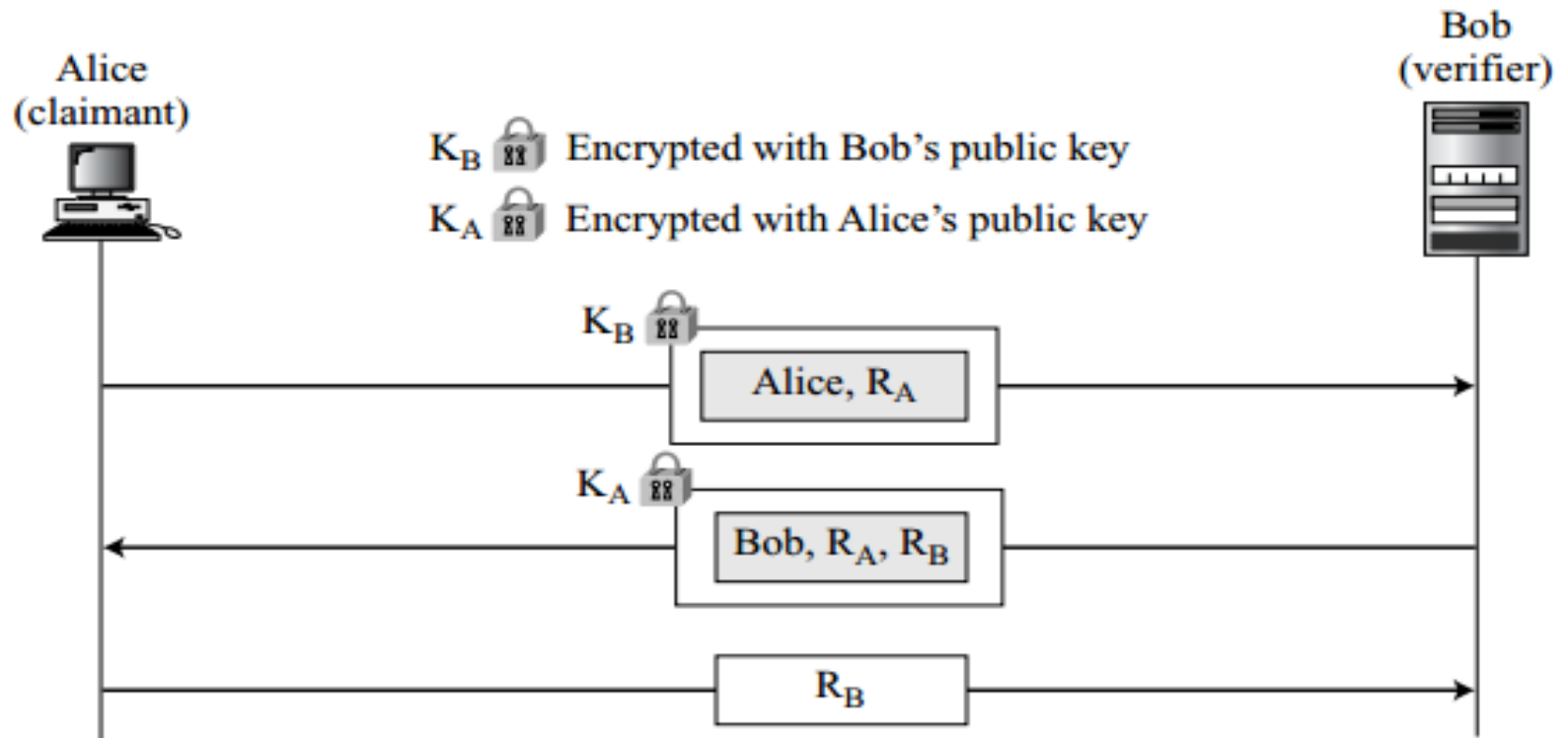
- Here the secret must be the private key of the claimant.
 - The verifier must encrypt the challenge using the public key of the claimant.
 - claimant then decrypts the message using her private key
 - The response to the challenge is the decrypted challenge.
- Two approaches:
 - unidirectional authentication
 - bidirectional authentication

Unidirectional asymmetric-key authentication



Bidirectional asymmetric-key authentication

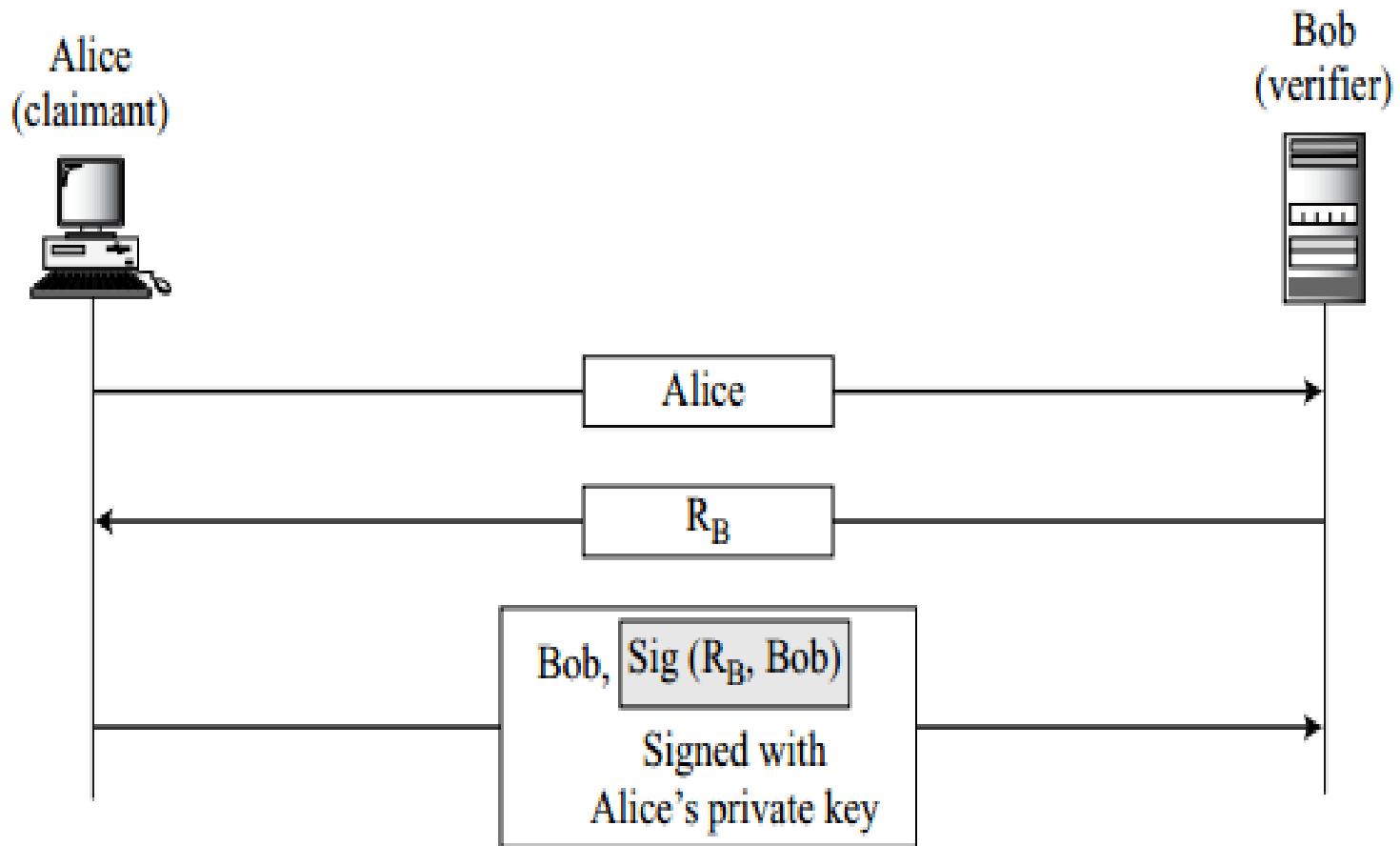
- Two public keys are used, one in each direction.



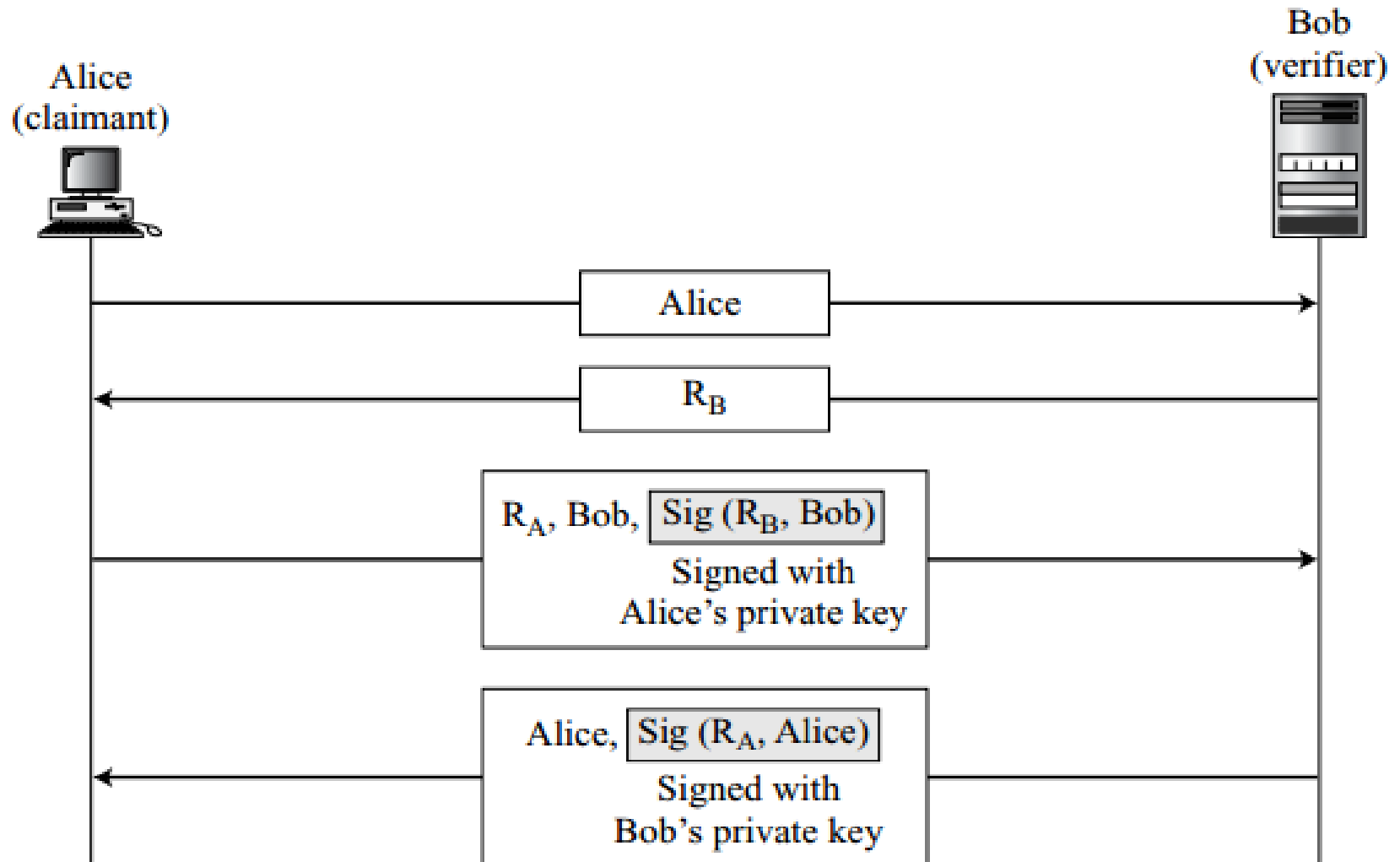
Challenge-response authentication using Digital Signature

- When a digital signature is used for entity authentication, the claimant uses her private key for signing.
- Two approaches are used:
 - unidirectional authentication
 - bidirectional authentication

Digital signature, unidirectional authentication



Digital signature, bidirectional authentication



Zero-knowledge authentication

- In **zero-knowledge authentication**, the claimant proves that she knows a secret without revealing it.
- The interactions are designed such that they cannot lead to revealing or guessing the secret.
- After exchanging messages, the verifier only knows that the claimant does or does not have the secret (only yes/no)
- **Fiat-Shamir Protocol** is based on zero-knowledge authentication

Fiat-Shamir Protocol

- A trusted third party chooses two large prime numbers p and q to calculate the value of $n = p \times q$.
 - The value of n is made public; the values of p and q are kept secret.
- Alice, the claimant, chooses a secret number s between 1 and $n - 1$ and calculates $v = s^2 \bmod n$.
- She keeps s as her private key and registers v as her public key with the third party.

Fiat-Shamir Protocol

- Verification of Alice by Bob:
 1. Alice, the claimant, chooses a random number r between 0 and $n - 1$ (r is called the commitment). She then calculates the value of $x = r^2 \bmod n$; x is called the witness.
 2. Alice sends x to Bob as the witness.
 3. Bob, the verifier, sends the challenge c to Alice. The value of c is either 0 or 1.
 4. Alice calculates the response $y = rs^c$. Note that r is the random number selected by Alice in the first step, s is her private key, and c is the challenge (0 or 1).
 5. Alice sends the response to Bob to show that she knows the value of her private key, s . She claims to be Alice.
 6. Bob calculates y^2 and xv^c . If these two values are congruent, then Alice either knows the value of s (she is honest) or she has calculated the value of y in some other ways (dishonest) because we can easily prove that y^2 is the same as xv^c in modulo n arithmetic as shown below:

$$y^2 = (rs^c)^2 = r^2 s^{2c} = r^2 (s^2)^c = xv^c$$

Fiat-Shamir Protocol

